



EXPERIMENT NO: 9

Student Name: Tanisha Kumari
Branch: BE-CSE
Semester: 6th
Subject Name: System Design

UID: 23BCS12542
Section/Group: KRG-1B
Date of Performance: 16/04/2026
Subject Code: 23CSH-314

1. Aim:

To design a highly scalable ticket booking platform that allows users to discover and purchase tickets for entertainment events, ensuring high availability for event discovery and strict consistency for seat reservations.

2. Objective:

- Understand the architecture of a massive-scale ticketing and reservation system.
- Identify functional requirements such as event discovery and real-time seat mapping.
- Identify non-functional requirements including handling 100 Million DAU.
- Analyze the decoupling of payment workflows and status updates using Kafka.
- Design REST APIs for event metadata, seat layouts, and secure transaction.

3. Functional Requirements (FR's)

- **Search & Discovery:** Users must search for events by title, city, or genre with fuzzy matching.
- **Seat Selection:** Real-time visualization of seat maps with current availability status.
- **Temporary Seat Locking:** Atomic locking of selected seats for 5 minutes to prevent race conditions.
- **Booking & Payment:** Secure transaction processing with automated ticket generation upon success.
- **Async Notifications:** Delivery of booking confirmation via Email/SMS post-transaction.

4. Non-Functional Requirements (NFR's)

- **High Availability:** The browsing/searching flow must be 99.99% available.
- **Strong Consistency:** Mandatory for the booking and seat-locking flow to prevent double bookings.
- **Low Latency:** Search results should return in <100ms; seat locking should occur in <200ms.
- **Scalability:** Ability to handle massive traffic spikes (100k+ TPS) during popular event releases.
- **Fault Tolerance:** Use of circuit breakers for third-party payment gateways to prevent cascading failures.

5. API Design

- GET /v1/events?city={id}&q={query}: Returns filtered events from ElasticSearch.
- GET /v1/shows/{show_id}/seats: Fetches real-time seat map from Read Replicas/-Cache.
- POST /v1/seats/lock: (Payload: seat_ids[]) Initiates Redis SETNX with a 300s TTL.
- POST /v1/bookings: Creates a pending order and initiates the payment redirect.
- POST /v1/payments/webhook: Asynchronous callback endpoint for payment gateway confirmation.

6. Core Entities

- **User:** Profile, authentication credentials, and preferences.
- **Event/Show:** Movie/Concert metadata, venue mapping, and timing.
- **Seat:** Physical coordinate, category (VIP/Gold), and price.
- **Booking/Order:** Transactional record linking User, Show, and Seats.
- **SeatLock:** Temporary distributed lock entity (Redis-backed).

7. Database Schema Design

- **User Table:** (user_id, name, email, phone)
- **Show Table:** (show_id, event_id, venue_id, start_time, base_price)
- **Seat Table:** (seat_id, show_id, seat_num, status [Available, Locked, Booked])
- **Booking Table:** (booking_id, user_id, show_id, total_amt, status [Pending, Paid, Failed])

8. High Level Design (HLD)

The architecture follows a microservices pattern optimized for high-concurrency:

- **Global Edge:** CDN handles static assets; L7 Load Balancer manages traffic across API Gateways.
- **Concurrency Layer:** Redis SETNX acts as a distributed lock. If payment fails or the 5-min TTL expires, the lock is released automatically.
- **Search Strategy:** A CDC (Change Data Capture) pipeline via Kafka Connect streams SQL updates to ElasticSearch for sub-second searching.
- **Reliability:** The Transactional Outbox Pattern ensures that the Booking DB update and the Notification Service (Kafka) are always in sync.
- **Data Strategy:** Write-intensive booking operations hit the Primary SQL DB, while heavy seat-map reads are distributed across Read Replicas.

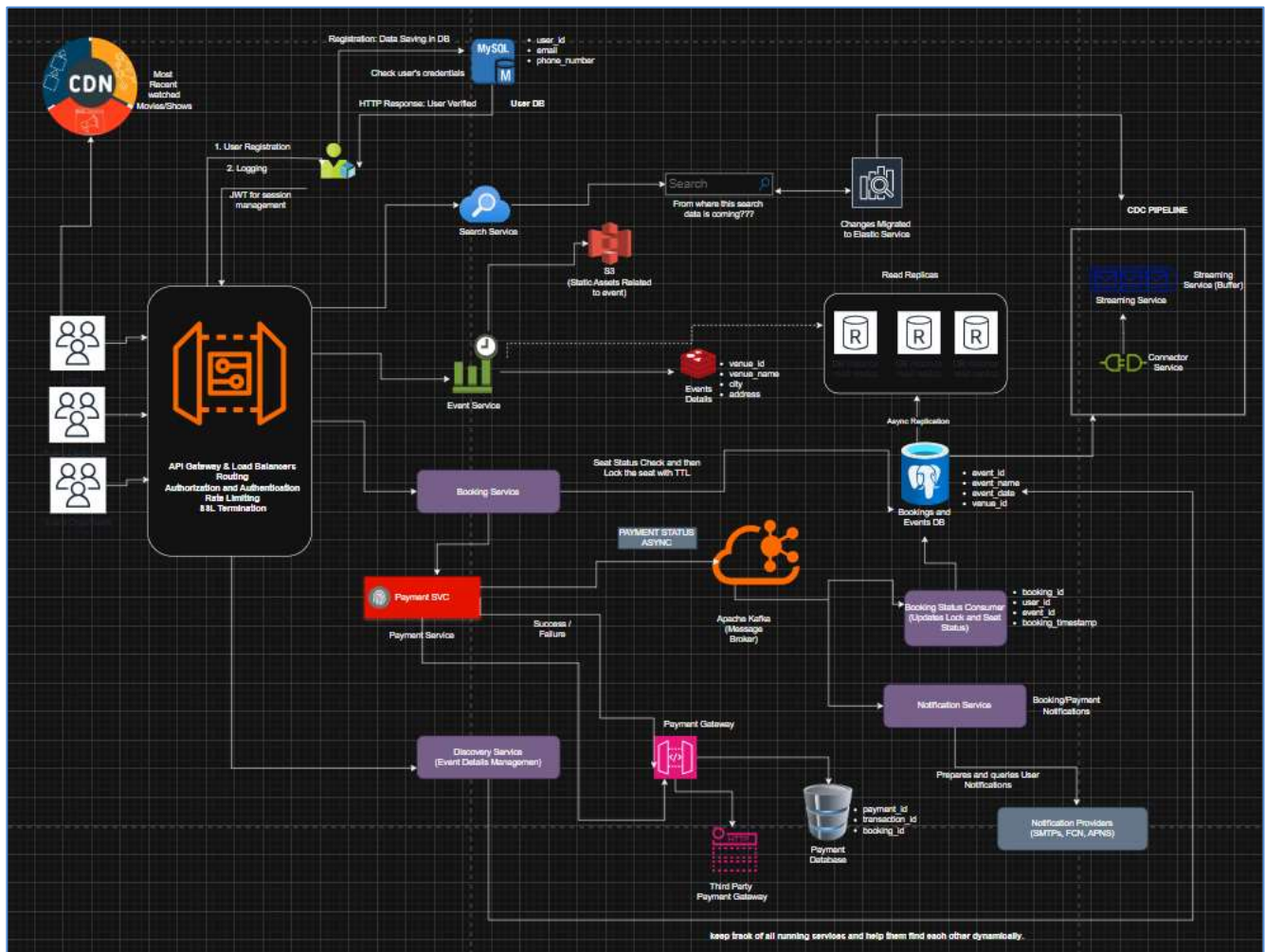


Figure 1: High Level Design of the Scalable Ticket Booking System